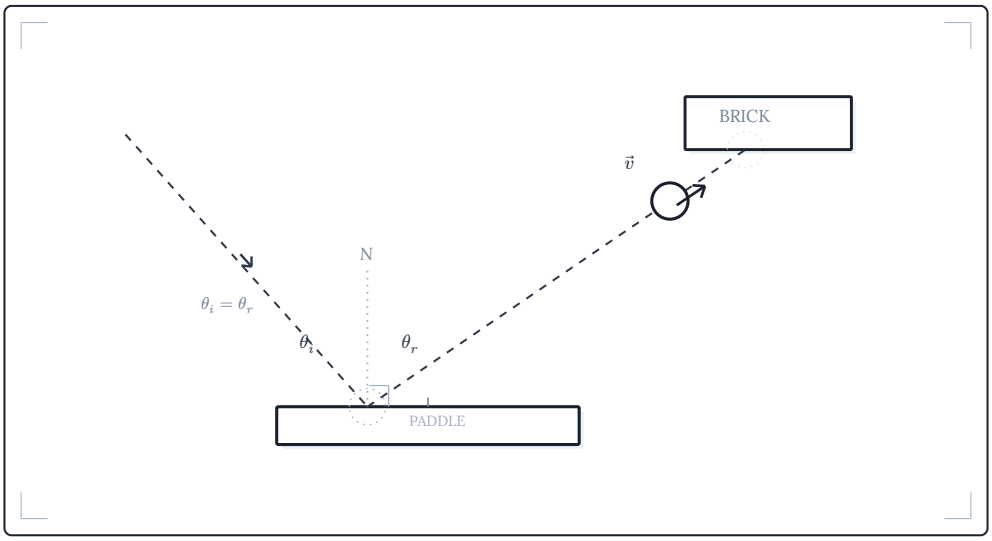
دانشگاه تهران
دانشکده علوم کامپیوتر

برنامه‌سازی پیشرفته

فاز پروژه

۱ه هسته بازی Breakout

اهداف، نیازمندی‌های فنی، طراحی شی‌گرا و معیارهای ارزیابی



«در سال ۱۹۷۶، نولان باشنل در شرکت آتاری از استیو جابز خواست بازی Breakout را با کمترین تعداد تراشه سخت‌افزاری بسازد. جابز پروژه را به استیو وازنیاک سپرد؛ وازنیاک طی ۴ شبانه‌روز کار بی‌وقفه، مدار بازی را تنها با ۴۴ تراشه پیاده کرد. ابتکارات وازنیاک در بهینه‌سازی مدارهای گرافیکی این بازی، مستقیماً الهام‌بخش او در طراحی معماری نمایش تصویر و رنگ در رایانه انقلابی Apple II شد.»

پرامپت پیشنهادی برای بازتولید تصویر خطی گیم‌پلی

Minimalist black and white technical line art of 1976 Breakout game mechanics, single paddle at bottom, single brick at top, dashed trajectory line showing angle of reflection off paddle, and single ball right before impact with brick, clean vector geometry, high contrast.

فهرست مطالب

قوانین و شرایط تحویل ۲
استقلال در پیاده‌سازی و مالکیت کد ۲
ثبت پیوسته در گیت‌هاب ۲
جلسه دفاع و گفت‌وگوی فنی ۲
بستر تحویل و موعد نهایی ۲
استفاده از ابزارهای هوش مصنوعی ۲
معرفی بازی و اهداف ۳
اهداف اصلی فاز اول ۳
اقدامات پیش از شروع ۴
۱. مخزن گیت‌هاب، گیت‌ایگنور و کامیت‌ها ۴
۲. تجربه بازی و درک رفتار فیزیکی ۴
۳. مدل‌سازی شی‌گرا و نقشه کلاس‌ها ۴
ساختار و معماری سیستم ۵
جریان داده در ۳ گام ۵
قرارداد ذخیره رکورد: `HighScoreStorage` ۵
موجودیت‌ها و هندسه بازی ۶
سازوکار گیم‌پلی و وضعیت‌ها ۷
گرافیک و واسط کاربری ۸
۱. دستگاه مختصات و هندسه بوم ۸
۲. پایداری تصویر و بافر دوگانه ۸
۳. اجزای دیداری و نوار وضعیت ۸
فیزیک و برخوردها ۹
۱. بازتاب زاویه‌دار از سکو ۹
۲. تشخیص برخورد با آجرها ۹
گام‌های پیاده‌سازی ۱۰
راهبرد توسعه و کامیت‌ها ۱۰
معیارهای ارزیابی و دفاع ۱۱
جلسه دفاع و گفت‌وگوی فنی ۱۱
بخش‌های امتیازی و اختیاری ۱۱

قوانین و شرایط تحویل

پروژه درس برنامه‌سازی پیشرفته یک ماجراجویی فردی و جذاب است تا آموخته‌های شی‌گرایی را در قالب یک نرم‌افزار واقعی تجربه کنید. برای اینکه این مسیر یادگیری لذت‌بخش و عادلانه باشد، چند اصل ساده و حرفه‌ای را با هم مرور می‌کنیم:

استقلال در پیاده‌سازی و مالکیت کد

این پروژه کاملاً **فردی** است و هدفش ساختن اعتمادبه‌نفس فنی شماست. تمام کدی که می‌نویسید باید حاصل درک مستقیم خودتان باشد. مطالعه منابع، راهنماها و مقالات برای یادگیری بسیار خوب است؛ اما کپی‌کردن کد دیگران یا ارسال کدهای ماشینی بدون تسلط کامل، با اهداف آموزشی درس در تضاد است. در جلسه دفاع باید بتوانید فلسفه پشت هر خط کد را با تسلط کامل توضیح دهید.

ثبت پیوسته در گیت‌هاب

فرآیند تفکر مهندسی شما از طریق کامیت‌های مرحله‌به‌مرحله[۱] در مخزن گیت‌هاب[۲] سنجیده می‌شود. کدنویسی یک جریان آرام و پیوسته است؛ بنابراین قابلیت‌ها را گام‌به‌گام اضافه کرده و با پیام‌های تمیز ثبت کنید. از ارسال ناگهانی کل پروژه در قالب یک کامیت در ساعات پایانی بپرهیزید.

جلسه دفاع و گفت‌وگوی فنی

پس از تحویل پروژه، در یک مصاحبه حضوری دوستانه با دستیاران آموزشی، کد شما بررسی می‌شود. در این نشست، تسلط شما بر معماری، کلاس‌ها و نحوه پیاده‌سازی ارزیابی شده و نمره نهایی پروژه تثبیت می‌گردد.

بستر تحویل و موعد نهایی

برای تحویل نهایی، نشانی مخزن گیت‌هاب پروژه را پیش از پایان موعد مقرر در سامانه کوئرا[۳] بارگذاری نمایید. دقت کنید که مخزن عمومی باشد یا دسترسی آن برای تیم تدریس باز شده باشد.

استفاده از ابزارهای هوش مصنوعی

هوش مصنوعی مثل یک منتور صبور است که می‌توانید برای فهم عمیق‌تر مفاهیم، اشکال‌زدایی و مشورت در طراحی از آن کمک بگیرید. با این حال، تصمیم‌گیرنده نهایی و مهندس اصلی سیستم شما هستید و تمام منطق بازی باید تحت تسلط کامل شما باشد.

همراهی دستیاران آموزشی

ما همیشه کنارتان هستیم تا مسیر یادگیری برایتان دلنشین‌تر شود. اگر در هر کجای معماری یا پیاده‌سازی با چالش یا ابهامی روبه‌رو شدید، در تالار گفتگو و جلسات رفع اشکال منتظر پرسش‌هایتان هستیم.

---

[۱] Git Commits
[۲] GitHub
[۳] Quera

معرفی بازی و اهداف

در سال ۱۹۷۶، شرکت آتاری با ساخت بازی تعاملی Breakout نقطه عطفی در تاریخ بازی‌های ویدئویی ساخت. داستان جذاب پشت این بازی، هنرنمایی استیو وازنیاک[۴] بود که مدار سخت‌افزاری آن را تنها با ۴۴ تراشه منطقی بست؛ نبوغی که بعدها مستقیماً الهام‌بخش او در طراحی معماری تصویر و رنگ رایانه جریان‌ساز اپل دو[۵] شد.

امروز Breakout از دوست‌داشتنی‌ترین پروژه‌ها برای لمس واقعی مهندسی نرم‌افزار شی‌گرا و سیستم‌های تعاملی است: یک سکوی چابک در پایین صفحه، توپی پرانرژی که میان دیواره‌ها کمانه می‌کند، و دیواری از آجرهای رنگارنگ که منتظر نشانه‌گیری دقیق شما هستند!

اهداف اصلی فاز اول

در این فاز می‌خواهیم با هم یک هسته تمیز، پایدار و استاندارد به زبان جاوا[۶] بسازیم. مهم‌ترین مهارت‌هایی که در این مسیر تمرین می‌کنید:

- **طراحی شی‌گرا و تفکیک دغدغه‌ها[۷]:** جدا نگه‌داشتن دنیای ریاضی و قوانین برخورد بازی از دنیای نقاشی و رندرینگ گرافیکی.
- **اصل تک‌مسئولیتی[۸]:** سپردن یک وظیفه مشخص و تمیز به هر کلاس، و پرهیز از ساختن کلاس‌های هیولایی همه‌کاره[۹].
- **کپسوله‌سازی شایسته[۱۰]:** مخفی‌کردن متغیرهای وضعیت درونی و کنترل تغییرات صرفاً از طریق توابع با نام‌گذاری شفاف.
- **مدیریت روان رخدادها[۱۱]:** گوش‌دادن بی‌درنگ به کلیدهای صفحه‌کلید و نوسازی پیوسته فریم‌ها در حلقه اصلی بازی.
- **معماری منعطف برای آینده:** طراحی کلاس‌ها به شکلی که در فاز بعد بتوان ذخیره رکورد در فایل و انواع پاورآپ‌ها را مثل آب خوردن به آن اضافه کرد.

محدوده مورد انتظار در فاز اول

خیالتان راحت باشد! هدف این فاز سلامت ساختار کد، فیزیک درست برخوردها و روانی گیم‌پلی است. نیازی به منوهای پیچیده، صداگذاری، یا گرافیک‌های سه‌بعدی نیست؛ یک واسط بصری ساده، تمیز و روان بالاترین نمره را برای شما به همراه خواهد داشت.

---

[۴] Steve Wozniak
[۵] Apple II
[۶] Java
[۷] SoC
[۸] SRP
[۹] God Class
[۱۰] Encapsulation
[۱۱] Event-Driven

اقدامات پیش از شروع

پیش از نوشتن نخستین خطوط کد در جاوا، ۳ گام راهبردی زیر مسیر طراحی و توسعه را برایتان کاملاً روشن و سازمان‌یافته می‌کند:

۱. مخزن گیت‌هاب، گیت‌ایگنور و کامیت‌ها

پروژه حرفه‌ای با مدیریت نسخه استاندارد و ثبت تاریخچه تغییرات آغاز می‌شود. ۳ وظیفه کلیدی در این بخش:

ایجاد مخزن در گیت‌هاب GitHub Repository

ساخت یک مخزن مستقل در گیت‌هاب به عنوان بستر اصلی نگه‌داری کد، رهگیری تغییرات در طول زمان و تحویل نسخه نهایی جهت ارزیابی در سامانه کوئرا.

پیکربندی گیت‌ایگنور `.gitignore`

فیلتر کردن فایل‌های خروجی کامپایل با پسوند `.class`، فایل‌های موقت سیستم‌عامل و تنظیمات محیط توسعه[۱۲] تا مخزن سبک، خوانا و عاری از زباله‌های باینری بماند.

کامیت‌های معنادار و استاندارد Conventional Commits

ثبت پیوسته و گام‌به‌گام پیشرفت با پیشوندهای استاندارد مانند `feat:`، `fix:` و `refactor:` برای شفاف‌سازی سیر تفکر مهندسی شما.

۲. تجربه بازی و درک رفتار فیزیکی

برای درک عمیق مکانیک‌ها و ریتم بازی، چند دقیقه یک نسخه اصیل بریک‌اوت را تجربه کنید و این ۳ مشخصه را مد نظر قرار دهید:

زاویه کمانه توپ از سکو

توپ در برخورد با مرکز سکو بازتابی متمایل به عمود دارد و با نزدیک‌شدن نقطه برخورد به گوشه‌ها، زاویه پرتاب تندتر و افقی‌تر می‌شود.

روانی شبیه‌سازی در ۶۰ هرتز

پاسخ بی‌درنگ سکو به فشردن کلیدها و حرکت پیوسته و نرم توپ در سراسر صفحه بدون افت فریم، مکث یا پرش تصویری.

مکانیک اصابت و انهدام آجرها

حذف فوری آجر منهدم‌شده پس از اصابت، بازتاب لحظه‌ای توپ در صفحه و ثبت ۱۰۰ امتیاز به نفع بازیکن.

۳. مدل‌سازی شی‌گرا و نقشه کلاس‌ها

یک طراحی روشن پیش از کدنویسی، زمان پیاده‌سازی و اشکال‌زدایی را به حداقل می‌رساند:

ترسیم نقشه کلاس‌ها UML Class Diagram

پیشنهاد می‌کنیم پیش از آغاز کدنویسی، کلاس‌ها، فیلدهای خصوصی و متدهای عمومی را در قالب یک نمودار کلاسی[۱۳] روی کاغذ یا با ابزارهای نرم‌افزاری ترسیم کنید تا تفکیک وظایف شفاف باشد.

نقش در جلسه مصاحبه دفاع

تحویل فایل نمودار در این فاز اختیاری است؛ اما داشتن نقشه معماری تسلط و استقلال شما در تفکیک مسئولیت‌ها را در برابر پرسش‌های تیم تدریس به اثبات می‌رساند.

---

[۱۲] IDE: Integrated Development Environment

[۱۳] UML: Unified Modeling Language

ساختار و معماری سیستم

تصور کنید تمام بازی در یک کلاس غول‌آسای ۲۵۰۰ خطی نوشته شود: از سرعت توپ و برخورد با دیواره تا رنگ صورتی آجرها و شنونده کلید Space! شاید اولش ساده به نظر برسد، اما با اولین باگ نامرئی، کد تبدیل به کلافی سردرگم می‌شود. نجات‌بخش ما در این بازی، الگوی درخشان معماری مدل-نما-کنترل‌گر[۱۴] است که وظایف را به ۳ بخش مستقل تقسیم می‌کند:

لایه مدل Model | کلاس‌های منطق

حقیقت جهان بازی؛ مختصات توپ، سکو، آجرها و جان‌ها. کدهای کاملاً مستقل از پنجره و گرافیک با آزمون‌پذیری آسان.

لایه نما BreakoutCanvas | View

پرده نقاشی و تماشا؛ دریافت وضعیت از کنترل‌گر و ترسیم اشکال، آجرها و قلب‌های جان. بدون داشتن منطق بازی.

لایه کنترل‌گر GameEngine | Controller

مغز هماهنگ‌کننده؛ اجرای حلقه بازی در ۶۰ هرتز، دریافت فشرده‌شدن کلیدها، حل برخوردها و فرمان بازترسیم به نما.

جریان داده در ۳ گام

۱. **ورودی کاربر:** فشرده شدن کلیدهای جهت‌نما یا کلیدهای فاصله، P و R توسط کنترل‌گر دریافت می‌شود.
۲. **محاسبه در مدل:** کنترل‌گر متدهای حرکتی سکو و توپ را صدا زده و وضعیت جدید دنیای بازی را ثبت می‌کند.
۳. **بازترسیم نما:** کنترل‌گر پس از قطعی شدن محاسبات فیزیک، وضعیت تازه را برای ترسیم به بوم نقاشی می‌فرستد.

قرارداد ذخیره رکورد: HighScoreStorage

برای استقلال کامل بازی از نحوه ذخیره‌سازی، یک رابط ساده تعریف می‌کنیم تا در این فاز رکورد در رم بماند و در فاز آینده بدون دست زدن به موتور بازی، ذخیره در فایل متصل شود:

```
public interface HighScoreStorage {
    int loadHighScore();
    void saveHighScore(int score);
}
```

پیاده‌سازی موقت در حافظه InMemoryScoreStorage

نگه‌داری بالاترین رکورد درون متغیرهای موقت حافظه رم[۱۵] طی اجرای بازی در فاز اول.

پیاده‌سازی آینده در فایل FileScoreStorage

ذخیره‌سازی دائمی رکورد در فایل متنی پس از تدریس مبحث ورودی/خروجی[۱۶] در فاز بعد بدون تغییر در موتور بازی.

---

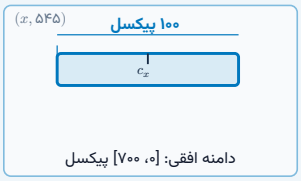
[۱۴] MVC: Model-View-Controller
[۱۵] RAM: Random Access Memory
[۱۶] I/O: Input/Output

موجودیت‌ها و هندسه بازی

جهان بازی Breakout از ۴ موجودیت بنیادین تشکیل شده که هر کدام مشخصات دقیق هندسی و مرزهای کپسوله‌شده داده‌ای دارند. مختصات صفحه با مبدا $(۰, ۰)$ در گوشه بالا-چپ بوم تعریف شده و تمام محاسبات بر پایه پیکسل انجام می‌گیرد:

سکو Paddle

ابعاد ۱۰۰×۱۵ px | Y=۵۴۵ px | بازه X: [۰, ۷۰۰]

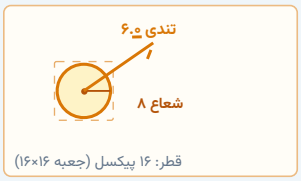


هندسه و ابعاد: مستطیل به ابعاد ۱۰۰ در ۱۵ پیکسل مستقر در فاصله ۴۰ پیکسلی از کف پنجره $(y = ۵۴۵)$. بازه مجاز جابه‌جایی افقی $x \in [۰, ۷۰۰]$ است تا سکو هرگز از حاشیه چپ یا راست بوم خارج نشود.

رفتار و کپسوله‌سازی: فیلدهای مختصات خصوصی (private) هستند. جابه‌جایی با سرعت ۸ پیکسل بر تیک صرفاً از طریق فراخوانی متدهای `moveLeft` و `moveRight` با اعمال مهار مرزی انجام می‌پذیرد.

توپ Ball

قطر ۱۶ px (شعاع ۸) | تندی پایه ۶.۰ px/tick

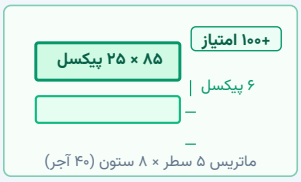


هندسه و ابعاد: دایره به قطر ۱۶ پیکسل با شعاع r برابر ۸ پیکسل. مستطیل محاطی تداخل آن ۱۶×۱۶ پیکسل است. بردار سرعت با تندی پایه ۶.۰ پیکسل بر تیک در برخورد با دیواره‌ها و سکو به صورت الاستیک بازتاب می‌یابد.

رفتار و کپسوله‌سازی: در شروع دور، توپ در نقطه مهار روی سکو $(x_{\text{paddle}} + ۴۲, ۵۲۹)$ قرار داشته و با کلید فاصله پرتاب می‌شود. فیلدهای سرعت و موقعیت خصوصی بوده و متدهای `reverseX` و `reverseY` جهت بردار را معکوس می‌کنند.

آجرها Brick

ماتریس ۴۰ آجر (۵×۸) | ابعاد ۸۵×۲۵ px | فاصله ۶ px



هندسه و ابعاد: شبکه ۵ سطر در ۸ ستون (۴۰ آجر). ابعاد هر آجر ۸۵ در ۲۵ پیکسل با فاصله ۶ پیکسل و حاشیه بالای ۶۰ پیکسل. فرمول مکان آجر سطر r و ستون c: $x = ۳۹ + c \times ۹۱$ و $y = ۶۰ + r \times ۳۱$.

رفتار و کپسوله‌سازی: هر آجر دارای مستطیل محیطی تداخل و فیلد وضعیت تخریب خصوصی است. با اصابت توپ، متد `hit` وضعیت آجر را به نابود تغییر داده، آجر از صفحه محو شده و ۱۰۰ امتیاز به بازیکن تعلق می‌گیرد.

نشانگر وضعیت و بوم HUD / Canvas

بوم ۸۰۰×۶۰۰ px | نوار وضعیت ارتفاع ۳۵ px

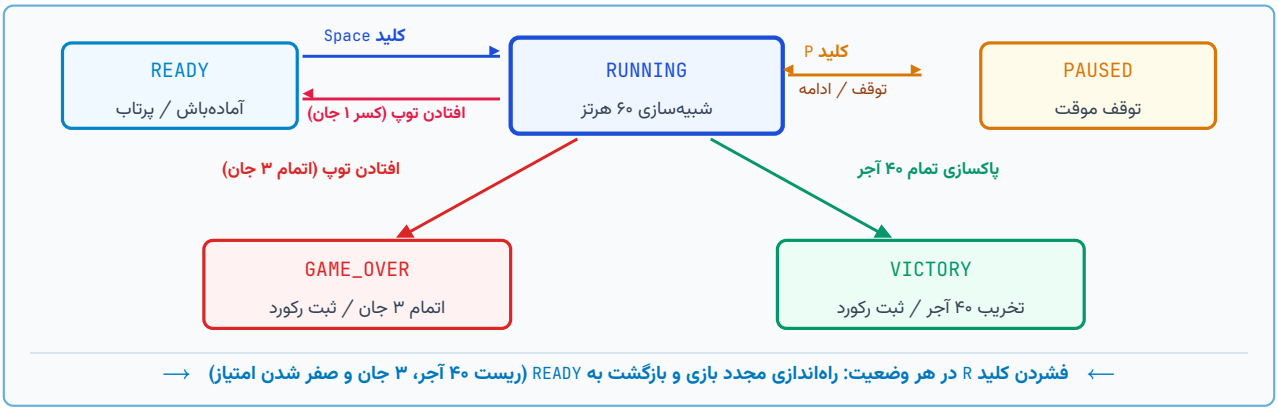


هندسه و ابعاد: پنجره گرافیکی با ابعاد ۸۰۰ در ۶۰۰ پیکسل و مبدا $(۰, ۰)$ در گوشه بالا-چپ. نوار بالایی صفحه به ارتفاع ۳۵ پیکسل برای نمایش وضعیت جان‌ها، امتیاز و رکورد بازی در نظر گرفته شده است.

رفتار و کپسوله‌سازی: موتور بازی وضعیت ۳ جان و امتیاز جاری را مدیریت می‌کند. خروج توپ از کف $(y > ۶۰۰)$ جان کسر کرده و پس از اتمام جان‌ها یا تخریب تمام ۴۰ آجر، پیام‌های پایان توسط نما نمایش داده می‌شوند.

سازوکار گیم‌پلی و وضعیت‌ها

مدیریت جریان بازی با یک ماشین وضعیت متناهی و شفاف پیاده‌سازی می‌شود تا واکنش بازی به کلیدها، برخوردها و رویدادهای مرگ و پیروزی کاملاً ساختاریافته و پیش‌بینی‌پذیر باشد:



وضعیت	رفتار، شرایط ورود و انتقال‌های مجاز در جریان بازی
حالت آماده‌باش `READY`	وضعیت نخستین بازی یا پس از هربار سوختن یک جان (در صورت داشتن جان باقی‌مانده). توپ روی مرکز سکو مستقر است و همراه با آن جابه‌جا می‌شود. با فشردن کلید فاصله، شبیه‌سازی شروع شده و بازی وارد وضعیت اجرا می‌شود.
حالت اجرای فعال `RUNNING`	حلقه اصلی شبیه‌سازی بلادرنگ ۶۰ هرتز؛ حرکت پیوسته توپ، فرمان‌پذیری سکو با کلیدهای چپ و راست، بازتاب از دیواره‌ها و انهدام آجرها. با سقوط توپ جان کم می‌شود و با تخریب تمام آجرها وضعیت پیروزی فعال می‌گردد.
توقف موقت `PAUSED`	با فشردن کلید P در حین بازی، محاسبات متوقف شده و پیام توقف روی صفحه ظاهر می‌شود. فشردن مجدد همین کلید شبیه‌سازی را دقیقاً از همان مختصات و با همان بردار سرعت پیشین ادامه می‌دهد.
پایان بازی / شکست `GAME_OVER`	با سقوط توپ از کف صفحه در شرایطی که هیچ جانی باقی نمانده رخ می‌دهد. شبیه‌سازی متوقف شده، پیام پایان بازی نمایش یافته و در صورت برتری امتیاز، بالاترین رکورد ثبت می‌شود. کلید R بازی را ریست می‌کند.
پیروزی بازیکن `VICTORY`	به محض انهدام کامل تمام ۴۰ آجر فعال می‌شود. حرکت توپ متوقف شده، پیام تبریک و امتیاز نهایی بازیکن نمایش داده شده و بالاترین رکورد ذخیره می‌گردد. فشردن کلید R امکان آغاز یک بازی جدید را فراهم می‌سازد.

پیاده‌سازی تمیز وضعیت‌ها با ساختار شمارشی (`enum`)

به جای تعریف متغیرهای بولی پراکنده و آسیب‌پذیر مانند `isPaused` یا `isGameOver`، حالت بازی را با یک داده شمارشی صریح (`enum GameState`) با ۵ مقدار بالا مدل‌سازی کنید. متد `update` در حلقه اصلی بازی تنها در وضعیت `RUNNING` محاسبات فیزیک و برخورد را پیش می‌برد و در سایر وضعیت‌ها شبیه‌سازی متوقف می‌ماند. این تفکیک از ایجاد باگ‌های منطقی متداول جلوگیری می‌کند.

گرافیک و واسط کاربری

نمایش بصری بازی بریک‌اوت بر پایه یک بوم دوبعدی شفاف و پایدار استوار است. رعایت چند اصل ساده گرافیک کامپیوتری باعث می‌شود بازی شما کاملاً روان و بدون پرش یا لرزش تصویر اجرا شود:

۱. دستگاه مختصات و هندسه بوم

در محیط‌های گرافیکی رایانه، جهت محورها با دستگاه دکارتی ریاضی کمی تفاوت دارد:

- نقطه مبدا با مختصات $(0, 0)$ در گوشه بالا-چپ بوم قرار گرفته است.
- محور افقی مثبت به سمت راست پنجره پیش می‌رود.
- محور عمودی مثبت به سمت پایین و کف پنجره امتداد دارد.

بنابراین هر زمان توپ رو به بالا و به سمت آجرها حرکت می‌کند، سرعت عمودی آن منفی است؛ و وقتی به سمت سکو در پایین سقوط می‌کند، سرعت عمودی مثبت است.

۲. پایداری تصویر و بافر دوگانه

شاید برایتان پیش آمده باشد که در برنامه‌های گرافیکی، تصویر چشمک بزند یا بلرزد. این پدیده به خاطر پاک‌شدن و کشیده‌شدن پیاپی اشیا روی صفحه است:

- با استفاده از بافر دوگانه[۱۷]، تمام نقاشی‌ها ابتدا در یک حافظه پس‌زمینه کامل می‌شوند و سپس فریم کامل‌شده یک‌باره به نمایش درمی‌آید.
- کتابخانه گرافیکی ACM این سازوکار را در بطن خود مدیریت می‌کند تا کار شما برای ایجاد انیمیشن روان و بی‌دردسر باشد.

۳. اجزای دیداری و نوار وضعیت

بخش بصری	مشخصات ترسیم و استانداردهای نمایش
بوم اصلی بازی	مستطیلی به ابعاد ۸۰۰ در ۶۰۰ پیکسل با پس‌زمینه تیره یا یکدست که محدوده شبیه‌سازی را مشخص می‌سازد.
سکو و توپ	سکو به صورت مستطیلی توپر با رنگ متمایز و توپ به شکل دایره‌ای یکدست با قطر ۱۶ پیکسل.
شبکه آجرها	مستطیل‌های رنگی هم‌اندازه با فاصله مرزی ۶ پیکسل؛ تنوع رنگ سطرها بازی را چشم‌نواز می‌کند.
نوار وضعیت[۱۸]	نوار بالای صفحه برای نمایش امتیاز، ۳ قلب جان و رکورد، به همراه پیام‌های توقف و پایان در وسط صفحه.

[۱۷] Double Buffering
[۱۸] HUD: Heads-Up Display

فیزیک و برخوردها

موتور فیزیک بازی وظیفه شبیه‌سازی حرکت پیوسته، محاسبه زاویه بازتاب از سکو و انهدام دقیق آجرها را بر عهده دارد:

۱. بازتاب زاویه‌دار از سکو

اگر توپ همیشه مثل آینه بازتاب کند، بازی سریعاً وارد یک چرخه تکراری و خسته‌کننده می‌شود. برای پویایی بیشتر، زاویه پرتاب با توجه به فاصله نقطه برخورد تا مرکز سکو تعیین می‌گردد:

روابط ریاضی کمانه زاویه‌دار

- **۱. انحراف نسبی برخورد نسبت به مرکز سکو ($d \in [-1.0, +1.0]$):** فاصله افقی تا مرکز (Δx) و نیم‌عرض سکو (w_{half}) را به‌صورت متغیرهای تفکیک‌شده تعریف می‌کنیم:

$$\Delta x = x_{\text{ball}} - x_{\text{paddle_center}} \quad , \quad w_{\text{half}} = \frac{W_{\text{paddle}}}{2}$$

$$d = \frac{\Delta x}{w_{\text{half}}}$$

مقدار منفی نشان‌دهنده اصابت در نیمه چپ ($\Delta x < 0$) و مقدار مثبت نشان‌دهنده اصابت در نیمه راست سکو ($\Delta x > 0$) است.

- **۲. زاویه پرتاب متناسب با نقطه برخورد (تا بیشینه ۶۰ درجه):**

$$\theta = d \times 60^{\circ}$$

برخورد در مرکز سکو بازتابی عمودی ($\theta = 0$) و برخورد در لبه‌ها ($d = \pm 1.0$) تندترین زاویه پرتاب را می‌سازد.

- **۳. پایستگی تندی کل توپ بر پایه قضیه فیثاغورس:**

$$v = \sqrt{v_x^2 + v_y^2}$$

- **۴. تجزیه سرعت به مؤلفه‌های جدید (پرتاب رو به بالا):**

$$v_{x'} = v \sin(\theta) \quad , \quad v_{y'} = -|v \cos(\theta)|$$

علامت منفی در مؤلفه عمودی تضمین می‌کند که توپ همیشه رو به بالا و به سمت آجرها کمانه کند.

```
// Calculate horizontal offset and half-width as separate variables
double paddleCenter = paddle.getX() + paddle.getWidth() / 2.0;
double ballCenter = ball.getX() + ball.getRadius();
double deltaX = ballCenter - paddleCenter;
double halfWidth = paddle.getWidth() / 2.0;

double offset = deltaX / halfWidth;
offset = Math.max(-1.0, Math.min(1.0, offset));

// Calculate bounce angle (max 60 deg) and preserve total velocity
double angle = offset * Math.toRadians(60.0);
double speed = Math.hypot(ball.getVx(), ball.getVy());

// Apply new velocity components (always deflect upwards)
ball.setVx(speed * Math.sin(angle));
ball.setVy(-Math.abs(speed * Math.cos(angle)));
```

۲. تشخیص برخورد با آجرها

برای بررسی برخورد توپ با آجرها و دیواره‌ها، از الگوریتم تداخل مستطیل‌های محیطی[۱۹] استفاده می‌کنیم. به محض برخورد، سرعت عمودی یا افقی معکوس شده، آجر مربوطه ناپدید می‌شود و ۱۰۰ امتیاز به بازیکن اضافه می‌گردد.

نکات طلایی برای پیاده‌سازی روان فیزیک

۱. **گیر افتادن توپ در سکو:** اگر فقط علامت سرعت را منفی کنید، توپ ممکن است در فریم بعدی همچنان داخل سکو بماند و شروع به لرزیدن کند! چاره کار: موقعیت عمودی توپ را پس از برخورد، مماس با سطح بالایی سکو قرار دهید.

۲. **تقسیم صفر ناشی از اعداد صحیح:** در محاسبه نسبت فاصله، متغیرها باید از نوع اعشاری `double` باشند؛ در غیر این صورت تقسیم عدد صحیح حاصل را صفر می‌کند.

۳. **پرش شبح‌وار از موانع[۲۰]:** اگر سرعت توپ خیلی زیاد شود، ممکن است در یک فریم از عرض آجر رد شود بدون اینکه برخوردی ثبت شود! سرعت را در اندازه‌ای منطقی نگه دارید.

---

[۱۹] AABB: Axis-Aligned Bounding Box

[۲۰] Tunneling

گام‌های پیاده‌سازی

برای اینکه قدم‌به‌قدم و با آرامش پیش بروید، پیشنهاد می‌کنیم ساخت بازی را به ۵ مرحله مستقل و لذت‌بخش تقسیم کنید:

۱. **گام اول: پنجره، بوم و سکوی بازی**
راه‌اندازی پنجره به ابعاد ۸۰۰ در ۶۰۰ پیکسل، ساخت کلاس سکو و کنترل حرکت افقی با کلیدهای چپ و راست.
معیار پذیرش: سکو به نرمی حرکت کند و با رسیدن به لبه‌های چپ و راست صفحه متوقف شود.

۲. **گام دوم: کلاس توپ، حرکت و بازتاب از دیواره‌ها**
ساخت کلاس توپ، بردار سرعت و بازتاب الاستیک از سه دیواره بالا، چپ و راست بوم.
معیار پذیرش: توپ در ابتدا همراه سکو حرکت کند و با زدن کلید فاصله، شلیک شود.

۳. **گام سوم: بازتاب زاویه‌دار از سکو و سیستم ۳ جان**
فرمول بازتاب زاویه‌دار (بین منفی ۶۰ تا مثبت ۶۰ درجه) و کسر جان با عبور توپ از کف صفحه.
معیار پذیرش: با افتادن توپ، یک قلب کم شود و توپ دوباره روی سکو آماده پرتاب باشد.

۴. **گام چهارم: ماتریس ۴۰ آجر، تشخیص برخورد و امتیاز**
چیدمان آجرها در ۵ سطر و ۸ ستون، الگوریتم برخورد و اضافه شدن ۱۰۰ امتیاز به ازای هر آجر.
معیار پذیرش: اصابت به هر آجر بلافاصله آن را محو کرده و جهت توپ را تغییر دهد.

۵. **گام پنجم: ماشین وضعیت، توقف، برد، باخت و شروع دوباره**
اتصال وضعیت‌های بازی، پیام‌های توقف با کلید P و راه‌اندازی دوباره در هر لحظه با کلید R.
معیار پذیرش: بازی با از دست رفتن جان‌ها یا تخریب تمام آجرها به پایان برسد و کلید R همه‌چیز را ریست کند.

راهبرد توسعه و کامیت‌ها

پیشنهاد می‌کنیم بعد از تکمیل هر گام، بلافاصله یک کامیت استاندارد در گیت‌هاب ثبت کنید. این کار به شما اطمینان می‌دهد که همیشه یک نسخه پایدار و کارآمد در جیب دارید!

معیارهای ارزیابی و دفاع

فاز اول پروژه دارای ۱۰۰ نمره تخصصی است که معادل ۲ نمره از ۲۰ نمره کل پایانی درس محاسبه می‌شود:

معیار ارزیابی	مؤلفه‌ها و جزئیات بررسی فنی	بارم
درستی عملکرد و حس بازی	حرکت روان ۶۰ هرتز، پرتاب دستی با کلید فاصله، بازتاب زاویه‌دار از سکو، بازتاب الاستیک از سه دیواره، تخریب دقیق آجرها با الگوریتم تداخل مستطیل‌ها، چرخه ۳ جان و ماشین وضعیت جامع.	۴۰ نمره
معماری و طراحی شی‌گرا	تفکیک دقیق سه لایه مدل، نما و کنترل‌گر، کپسوله‌سازی کامل فیلدها با سطح دسترسی خصوصی، پایبندی به اصل تک‌مسئولیتی، و بکارگیری رابط قرارداد ذخیره‌سازی داده.	۳۵ نمره
کیفیت کد و مهندسی نرم‌افزار	پرهیز کامل از اعداد جادویی و تجمیع ثابت‌ها در یک کلاس مرجع، نام‌گذاری گویای متغیرها و توابع، ساختار استاندارد بسته‌بندی و تعهد به کامیت‌های منظم در گیت‌هاب.	۱۵ نمره
تسلط و گفت‌وگو در جلسه دفاع	تشریح روان محاسبات هندسی زاویه کمانه، تبیین منطق تشخیص برخورد، توجیه تصمیم‌های شی‌گرایی و پاسخ به پرسش‌های مفهومی دستیاران آموزشی.	۱۰ نمره
مجموع کل ارزیابی	**ارزیابی جامع فاز اول هسته بازی بریک‌اوت**	**۱۰۰ نمره**

جلسه دفاع و گفت‌وگوی فنی

جلسه تحویل پروژه، یک مصاحبه خشک نیست؛ یک گفت‌وگوی صمیمی و دوستانه بین شما و دستیاران آموزشی است تا دستاوردها و تلاش‌هایتان را نشان دهید و بازخوردهای سازنده بگیرید. اگر کدها را خودتان نوشته باشید و چرایی تصمیم‌هایتان را بدانید، این جلسه بسیار لذت‌بخش خواهد بود:

- **تسلط کامل بر کدها:** اگر ساختار کدهای ارسالی را خوب بشناسید و منطق محاسبات فیزیک و کلاس‌ها را روان توضیح دهید، کل نمره را به سادگی دریافت خواهید کرد.
- **اصالت کار و استقلال:** هدف ما اطمینان از یادگیری واقعی شماست. اگر شخصی نتواند دلیل تصمیم‌های کدش را توضیح دهد یا تغییرات کوچکی را در جلسه اعمال کند، نمره تعدیل می‌شود. هوش مصنوعی دستیار خوبی برای یادگیری است، اما مهندس اصلی پروژه خود شمایید.

بخش‌های امتیازی و اختیاری

اگر دوست دارید مهارت‌های خود را فراتر از حد انتظار به چالش بکشید، این دو بخش امتیازی جذاب را امتحان کنید:

- **آجرهای مقاوم دوضربه‌ای (۱۰ نمره امتیازی):** آجرهایی در سطرهای بالا که با ضربه اول ترک برمی‌دارند و با ضربه دوم منهدم می‌شوند (۲۰۰ امتیاز).
- **تعویض پویای راهبرد ذخیره‌ساز (۵ نمره امتیازی):** تغییر بلادرنگ نوع ذخیره رکورد میان حافظه موقت و فایل متنی در حین اجرای بازی.